Document number: P1869R0
Date: 2019-09-17
Audience: Library Evolution Working Group
Reply-to: *Tomasz Kamiński <tomaszkam at gmail dot com>*
Michał Dominiak <griwes at griwes dot info>

Rename `condition_variable_any` interruptible wait methods

1. Introduction

This paper proposes small improved to the `condition_variable_any` interruptible wait interface (accepting `stop_token`), that makes them more consistent with the rest of the standard and more ergonomic to use.

Before	**After**

`cv.wait_until(lock, [&cont] {`
`  return cont.empty();`
`}, stop_token);`
	`cv.wait_on(lock, stop_token, [&cont] {`
	`  return cont.empty();`
	`});`

`cv.wait_until(lock, time_point, [&cont] {`
`  return cont.empty();`
`}, stop_token);`
	`cv.wait_on_until(lock, stop_token, time_point, [&cont] {`
	`  return cont.empty();`
	`});`

`cv.wait_for(lock, 10s, [&cont] {`
`  return cont.empty();`
`}, stop_token);`
	`cv.wait_on_for(lock, stop_token, 10s, [&cont] {`
	`  return cont.empty();`
	`});`

2. Revision history

2.1. Revision 0

Initial revision.

3. Motivation and Scope

Currently, the `condition_variable_any` interruptible wait methods, i.e. ones that are accepting `stop_token`, are defined as follows:

```
template<class Lock, class Predicate>
  bool wait_until(Lock& lock, Predicate pred, stop_token stoken);
template<class Lock, class Clock, class Duration, class Predicate>
  bool wait_until(Lock& lock, const chrono::time_point<Clock, Duration>& abs_time
                  Predicate pred, stop_token stoken);
template<class Lock, class Rep, class Period, class Predicate>
  bool wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time,
                Predicate pred, stop_token stoken);
```

The paper proposes to adjust their names and signatures to make them more consistent with the rest of the "[thread] Thread support library", by using `wait_on` prefix and placing `stop_token` as the second argument (thus making `pred` last).

```
template<class Lock, class Predicate>
  bool wait_on(Lock& lock, stop_token stoken, Predicate pred);
template<class Lock, class Clock, class Duration, class Predicate>
  bool wait_on_until(Lock& lock, stop_token stoken, const chrono::time_point<Clock, Duration>& abs_time
                     Predicate pred);
template<class Lock, class Rep, class Period, class Predicate>
  bool wait_on_for(Lock& lock, stop_token stoken, const chrono::duration<Rep, Period>& rel_time,
                   Predicate pred);
```

3.1. Standard reserve's `_until` prefix for functions with absolute timeouts

In the [thread.req.timing] Timing specifications p4 the standard explicitly states that the `_until` prefix is used with functions that accept `time_point` as argument:

> The functions whose names end in `_until` take an argument that specifies a time point. These functions produce absolute timeouts. Implementations should use the clock specified in the time point to measure time for these functions.

Out of 13 function using `_until` suffix, that are defined in the [thread] Thread support library

1. `this_thread::sleep_until`,
2. `timed_mutex::try_lock_until`,
3. `recursive_timed_mutex::try_lock_until`,
4. `shared_timed_mutex::try_lock_until`,
5. `shared_timed_mutex::try_lock_until`,
6. `unique_lock::try_lock_until`,
7. `shared_lock::try_lock_until`,
8. `condition_variable::wait_until` (x2),
9. `condition_variable_any::wait_until` (x4).

only the `condition_variable_any::wait_until` method does not accept the `time_point` argument:

```
template<class Lock, class Predicate>
  bool wait_until(Lock& lock, Predicate pred, stop_token stoken);
```

This paper addresses above inconsistency and conflicting wording, by renaming the method to `wait_on`.

3.2. Predicate as last argument

Similarly to above, pre-existing non-interruptible waits methods, accept the predicate as the last argument (after lock and optional `time_point`/`duration` argument). This allows invocations, that uses lambda as the predicate, to be nicely formatted:

```
cv.wait_for(lock, 10s, [&cont] {
  return cont.empty();
});
```

However, the newly introduced interruptible waits, places the additional `stop_token` argument, after the predicate:

```
cv.wait_for(lock, 10s, [&cont] {
  return cont.empty();
}, stop_token);
```

This paper moves the `stop_token` to second the position, thus allowing:

```
cv.wait_on_for(lock, stop_token, 10s, [&cont] {
  return cont.empty();
});
```

4. Proposed Wording

The proposed wording changes refer to N4830 (C++ Working Draft, 2019-08-15).

Apply following changes to [thread.condition.condvarany] Class `condition_variable_any`:

```
// [thread.condvarany.intwait], interruptible waits
template<class Lock, class Predicate>
  bool wait_on~~until~~(Lock& lock, stop_token stoken, Predicate pred~~, stop_token stoken~~);
template<class Lock, class Clock, class Duration, class Predicate>
  bool wait_on_until(Lock& lock, stop_token stoken, const chrono::time_point<Clock, Duration>& abs_time
                     Predicate pred~~, stop_token stoken~~);
template<class Lock, class Rep, class Period, class Predicate>
  bool wait_on_for(Lock& lock, stop_token stoken, const chrono::duration<Rep, Period>& rel_time,
                   Predicate pred~~, stop_token stoken~~);
```

Apply following changes to [thread.condvarany.intwait] Interruptible waits:

```
template<class Lock, class Predicate>
  bool wait_on~~until~~(Lock& lock, stop_token stoken, Predicate pred~~, stop_token stoken~~);
```

```
[...]

   template<class Lock, class Clock, class Duration, class Predicate>
     bool wait_on_until(Lock& lock, stop_token stoken, const chrono::time_point<Clock, Duration>& abs_time
                    Predicate pred~~, stop_token stoken~~);

[...]

   template<class Lock, class Rep, class Period, class Predicate>
     bool wait_on_for(Lock& lock, stop_token stoken, const chrono::duration<Rep, Period>& rel_time,
                   Predicate pred~~, stop_token stoken~~);
```

Update the value of the `__cpp_lib_jthread` in table "Standard library feature-test macros" of [support.limits.general] to reflect the date of approval of this proposal.

5. Acknowledgements

Special thanks and recognition goes to Sabre (http://www.sabre.com) for supporting the production of this proposal and author's participation in standardization committee.

6. References

1. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4830, https://wg21.link/n4830)